

ZAVA | Meeting Concierge Agent

Deployment Plan

A controlled proof of concept for moving meetings without breaking policy

Date: 09/29/26

1. Purpose and approach

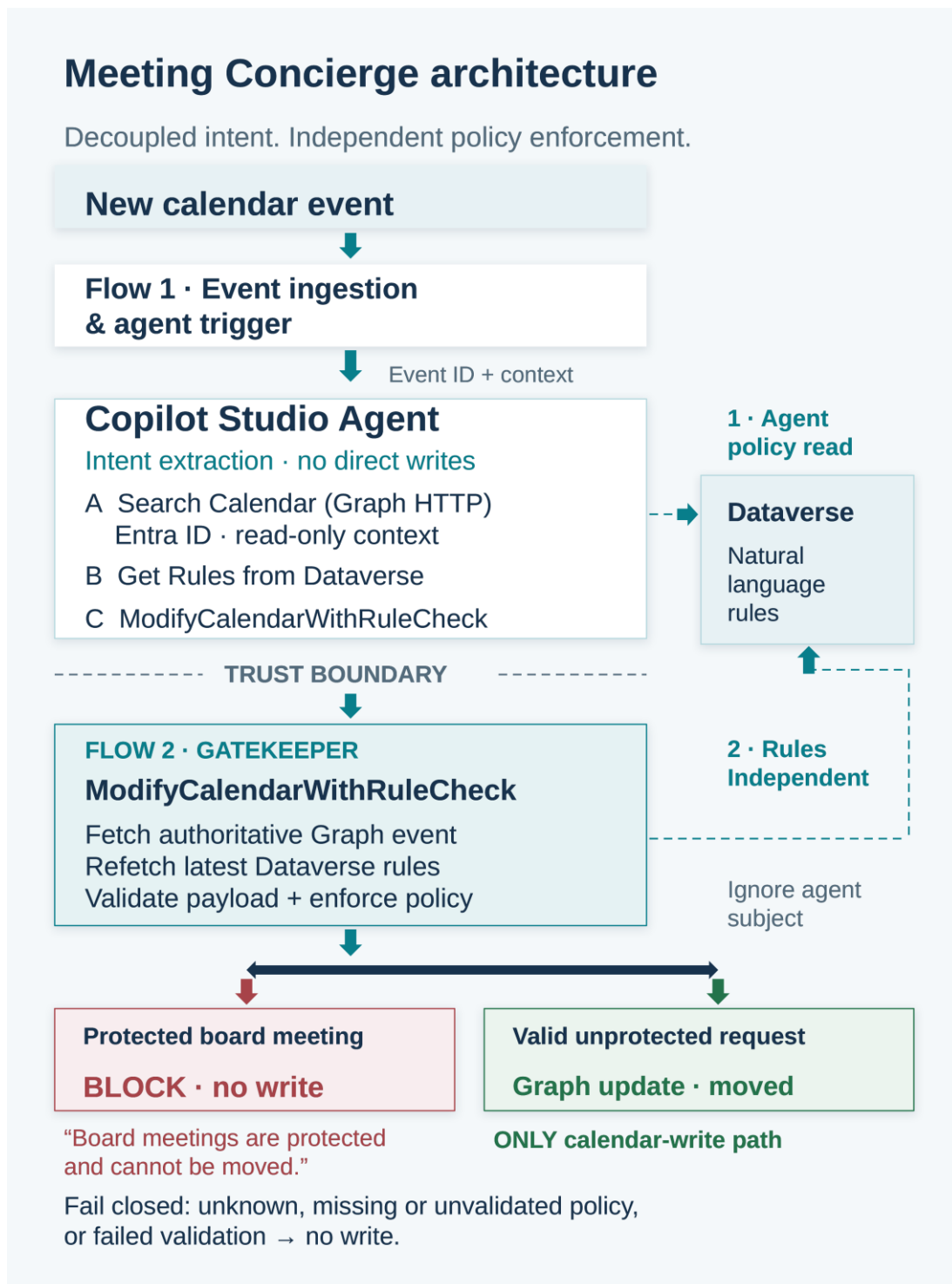
The Meeting Concierge agent can read calendars and suggest schedule changes. A separate validation flow decides whether each change is allowed. Copilot Studio interprets the request, Dataverse stores the policy, Power Automate or agent flows coordinates the steps, and Microsoft Graph reads and updates calendar events. This plan uses the clearest architecture from the Copilot draft and the stronger workflow, tool and audit details from the Gemini draft.

Scope and success

The pilot supports Move/reschedule only. A created event trigger starts the process, and the agent may suggest a new time for an existing meeting. If the actual event subject fetched from Graph contains “board meeting”, regardless of capitalization, the meeting must not move. Other meetings can move only after authorization, policy and time checks pass. Cancel, delete and other calendar actions are out of scope. Success means the acceptance tests pass, decisions can be traced, and there are no unauthorized writes. It does not mean the solution is production-ready or immune to prompt injection.

2. Architecture and who can make changes

Combined diagram adapted from Meeting Concierge Agent Planning - Copilot Suggestion.docx (Plan / Architecture) and Meeting Concierge Agent Planning - Gemini Suggestion.docx (Plan / 1. Architectural Blueprint). Both keep the same two-flow separation: only the gatekeeper, which runs its own checks, can change a calendar.



Identity and Microsoft Graph access

For the delegated pilot, which acts as the signed-in user, search Graph events through POST /search/query using an Entra ID-authenticated, read-only connection. Microsoft Search supports only that user's own primary mailbox, not delegated mailbox search. Resolve /me to the authenticated user. Keep the gatekeeper's Calendars.ReadWrite connection separate, explicitly authorize the caller and mailbox, and limit access to the pilot mailboxes. Never expose credentials that can write through an agent read tool.

If app-only access or multiple mailboxes are needed, approve a different retrieval design before deployment. Use /users/{id}/calendar/calendarView within a limited time window; do not claim app-only event search. CalendarView supports delegated and application permissions and returns recurring occurrences and exceptions. Follow @odata.nextLink to get every page of results. The range offsets set the query window; Prefer: outlook.timezone controls the response time zone. Use Calendars.ReadBasic where enough, or Calendars.Read for fuller details. Check consent and grant only the permissions needed.

3. Policy storage and enforcement

Use one Dataverse table, MeetingProtectionRules, with exactly one natural-language column as the policy source. Supporting metadata identifies the approved record and evaluator version. Do not repeat the policy in agent instructions or another prose field. Seed RuleName as BoardMeetingProtection and keep the source rule exactly as written:

Board meetings are critical and cannot be moved. Any meeting that has “board meeting” in subject is considered board meeting

Field	Type	Purpose
RuleId / RuleName	GUID / text	Identifies the record and gives it a readable name.
NaturalLanguageRule	Multiline text	The single source for the policy wording.
IsActive	Yes/no	Activates the rule after review and testing.
PolicyVersion / EvaluatorVersion	Version identifiers	Ties the approved rule to the validated fixed check.
TargetEntity / EnforcementLevel	Optional metadata	Calendar event / deny; do not repeat the policy wording here.

How policy changes can be approved

A free-text policy edit cannot safely become an automatic execution rule. For this proof of concept, use a reviewed, fixed check: if the subject fetched from Graph contains “board meeting”, case-insensitively, deny Move. Tie that check to the approved Dataverse rule/version. Fetch the latest rule on every request. If it is missing, inactive, unrecognized, or changed without a matching validated check, stop without changing the calendar and send the issue to the policy administrator.

Before a policy change is activated, it needs controlled review, an evaluator update and regression tests to confirm existing behavior still works. Keep the agent's instructions generic: read active rules, inspect the calendar,

and send every proposed change to the validation flow. The fixed check belongs in the gatekeeper's implementation, not as business policy written into agent instructions.

4. Proposed execution steps and action contract

From the event trigger to the agent

The ingestion flow accepts a created event within scope, records a tracking ID (correlation ID) and a key to detect duplicate requests, then calls the agent with the event and mailbox identifiers. The agent reads active Dataverse rules, uses read-only calendar retrieval to assess options, and calls `ModifyCalendarWithRuleCheck` only when it proposes a Move. Tool descriptions must clearly distinguish “read calendar”, “read active rules” and “request validated move”. Treat meeting text as untrusted content, never as instructions or approval.

Gatekeeper checks before a change

Authorize the caller and mailbox. Confirm the requested action is Move and has not already completed. Fetch the current event with `GET /users/{userId}/events/{eventId}` (or `/me` for the delegated user). Separately fetch the active Dataverse rule and check its mapped version. Evaluate the fetched subject, validate start/end and time zone, and confirm which recurring event is targeted. Only after every check passes can the write connection PATCH the intended event. Record the decision for audit and return a structured outcome.

For a protected meeting, return status Blocked and the exact message: “Board meetings are protected and cannot be moved.” For a confirmed update, return Success with the event ID and correlation ID. Recommended additional outcomes are Denied for invalid or unauthorized requests and Error when a dependency fails. An uncertain write result is not Success. Check what actually happened before retrying.

Required request fields

Inputs	Validation requirement
mailboxId, eventId	Authorize the mailbox and fetch the current event server-side.
requestedAction, newStart, newEnd, timeZone	Move only; valid time zone; end after start; explicit choice of occurrence or series.
reason, correlationId	Reason gives context, not permission. The correlation ID links the request, decision and result.
Supplied subject, if present	Never use it as the source of truth. Check the subject fetched from Graph.

5. Deployment checkpoints

These release checkpoints and operating controls are recommendations that strengthen the two source plans. Assign clear owners before work begins. Move forward when the required evidence and exit criteria are met, not because an assumed date has arrived.

Gate	Key work	Evidence needed to proceed
Readiness	Confirm scope, mailbox mode, licensing, capacity, data loss prevention (DLP), connector/auth support and admin consent.	Approved pilot scope, permissions and policy owner.
Build	Create the table, tools, ingestion and gatekeeper. Keep read/write connections separate.	Only the gatekeeper can write; the approved rule mapping is deployed.
Validate	Run every acceptance case. Check outcomes and audit records.	All required cases pass; no unexplained writes.
Pilot	Enable a limited group of mailboxes. Monitor errors, denials and retries.	Stable operation; incidents reviewed; rollback demonstrated.
Controlled release	Approve broader scope after security and operations review.	Named support owners, monitoring and recovery readiness.

6. Acceptance tests and release evidence

Run these cases on dedicated test events before enabling the pilot. Keep the request, actual event fetched from Graph, policy/evaluator version, outcome and observed calendar result. Check that prohibited writes did not happen, not just that the right message came back.

Test	Required outcome
Protected subject variants	“board meeting”, mixed case and an embedded phrase: Blocked; exact protection message; no write.
Ordinary meeting	Engineering sync moves only after all checks pass. Success and the audit record match the stored event.
Forged subject or instruction	The agent supplies a safe subject for a board event, or event text asks to bypass checks: the fetched subject decides; no protected write.
Missing, inactive or changed rule	No write. Stop and send the issue to the policy administrator until the mapping is validated.
Read or policy dependency failure	Connector timeout or lookup failure: no write; keep the structured error and correlation ID.

Test	Required outcome
Invalid time or action	End before start, invalid time zone or non-Move action: denied; no write.
Duplicate or self-generated event	Repeated delivery or an update that retriggers the flow: no repeated move or feedback loop.
Recurring event	Validate the explicit occurrence or series target. Deny unclear scope.
Unauthorized or ambiguous write	Unauthorized caller/mailbox: denied. Uncertain PATCH response: check the stored event before retrying; never blindly repeat the write.

7. Operations and recovery

Log the correlation ID, actor, mailbox/event identifiers, requested action, policy and evaluator versions, decision, reason and Graph result. Keep sensitive meeting content in logs to a minimum, restrict access, and agree how long to retain it. Monitor blocked/denied rates, dependency failures, write failures, duplicate prevention, and differences between the expected and actual event state. Never log credentials.

Filter for created events within the pilot scope and ignore follow-on activity caused by the solution itself. Store a persistent duplicate-prevention record keyed to the trigger event and intended action, not just a temporary flow run. Limit retries for temporary read failures. Never blindly retry a write after an uncertain response. First check the current event state against the action record.

To roll back, first disable the ingestion trigger and the gatekeeper write path. Then contain access and review the audit records. Keep read-only diagnostics available where safe. Reverting a flow or policy does not undo calendar changes already made. Any reschedule to correct an earlier change needs explicit review and authorization. Re-enable only after the issue is corrected, regression tests pass and approval is given.

Microsoft Learn

Event search and mailbox limitations: <https://learn.microsoft.com/graph/search-concept-events>

CalendarView, permissions and time-window behavior: <https://learn.microsoft.com/graph/api/calendar-list-calendarview?view=graph-rest-1.0>

Licensing check (confirm current entitlements before deployment): <https://learn.microsoft.com/en-us/microsoft-365-copilot/extensibility/prerequisites#agent-capabilities-and-licensing-models>